



AI Agents for Developers / How AI Agent Orchestration Works

How AI Agent Orchestration Works

A single AI agent ↗ can produce useful output on its own. Add a second agent, such as a reviewer, a test runner, or a documentation writer, and something has to decide which agent runs first, what context each one receives, what happens if one fails, and when the overall workflow is done.

That coordination layer is AI agent orchestration. It handles planning, execution routing, state management, inter-agent communication, retries, and workflow control, primarily in multi-agent systems ↗, though even a single-agent plan-and-execute loop benefits from explicit coordination logic. The orchestration layer governs workflow

execution, not how any single agent generates its output.

"On average, developers report that approximately 46% of the code they produce is fully generated by AI agents, 39% is written with AI assistance, and 27% is written entirely manually."

Source: Preliminary findings from the JetBrains Developer Ecosystem Survey 2026 • 15,000+ developers worldwide

This guide covers how orchestration works at a mechanical level: the execution loop, coordination patterns (including centralized, distributed, and hierarchical), runtime controls, and challenges that arise when multiple agents share state, and how to choose a strategy that matches your system's complexity and reliability targets. It stays at the conceptual level, focusing on how the coordination layer behaves rather than how any one framework implements it, and links out to dedicated guides on the adjacent topics it touches: agent architecture, memory, planning, and multi-agent design.

Table of content

What is AI agent orchestration?

How AI agent orchestration works

Core AI agent orchestration patterns

Orchestration in multi-agent systems

Common challenges in AI agent orchestration

Choosing the right AI agent orchestration strategy

What this means for developers

What is AI agent orchestration?

AI agent orchestration, also called AI workflow orchestration, is the coordination layer that translates a high-level objective into a sequence of executable steps, ensures those steps happen in the right order with the right context, and keeps the workflow from breaking down when something goes wrong. It connects LLMs, agents, tools, memory systems, and workflow logic into a functioning system.

That coordination spans components that would otherwise operate in silos: models with distinct capabilities, agents with specific roles, external tools, such as APIs, databases, and terminals, memory systems, such as short-term working memory and long-term vector stores, and the workflow logic that stitches them together.

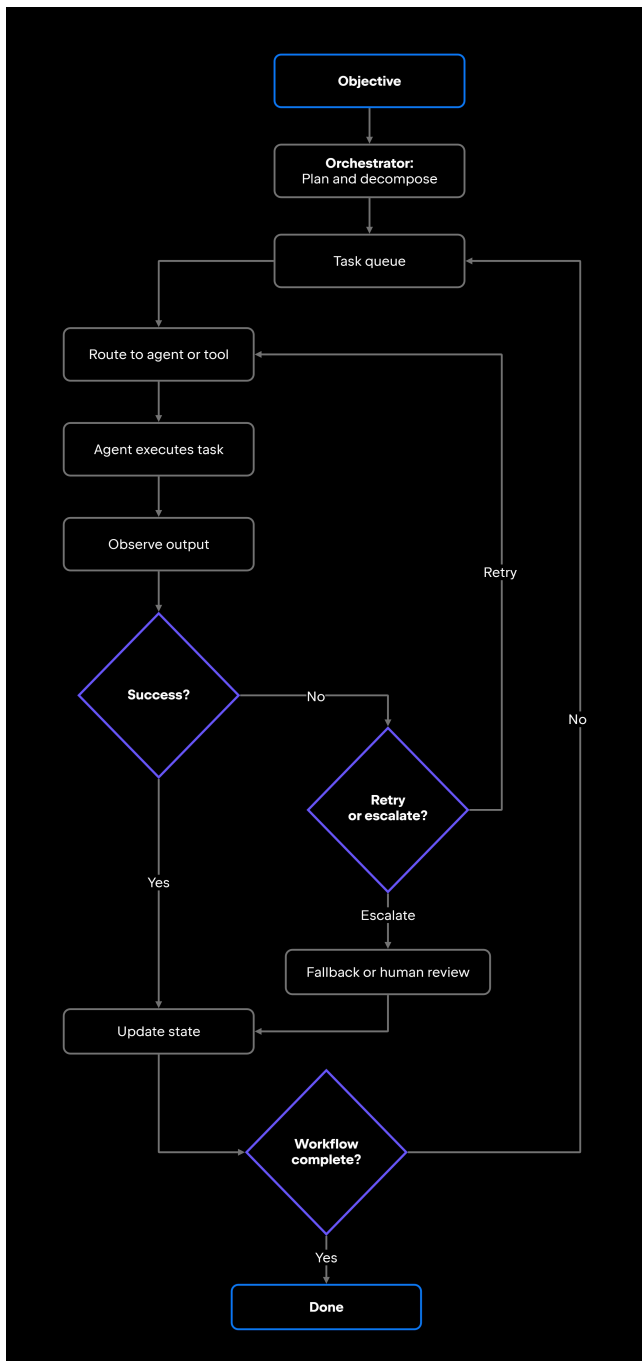
Orchestration works one level above inference. A model generates tokens; an agent calls tools; the orchestrator decides when each step happens, routes outputs to the right next step, tracks intermediate state, and handles failures. In frameworks like LangGraph⁷, orchestration is the runtime that holds everything together.

It applies to single-agent systems, too. Even a solo agent running a plan-and-execute loop needs

something to manage its iteration: tracking which subtasks are done, what memory to pass forward, and when to stop. That management, the loop the orchestrator actually runs, is what the next section unpacks.

How AI agent orchestration works

Underneath every orchestrated system is one control loop: plan, route, execute, observe, adapt. Understanding how AI agent orchestration works means following that loop, because the orchestrator is a workflow manager. It decides when each agent runs, whether to retry after an error, and when to hand results to the next step. An objective enters the system, is decomposed into a task queue, and the orchestrator evaluates each output before the workflow advances.



Each turn of the loop runs through the same core stages: planning, routing, state management, and feedback handling, which the following subsections will cover one at a time.

Planning and task decomposition

Most real objectives can't be executed in a single agent call. "Refactor the authentication module" involves reading existing code, identifying issues, writing changes, running tests, and generating a PR description. The orchestration layer decomposes this work into discrete, scheduled tasks.

Decomposition determines execution order and dependency relationships. Some tasks run sequentially because the output of one feeds the input of the next. Others can run in parallel, such as generating unit tests and writing documentation for the same feature. The orchestrator tracks these dependencies and schedules execution accordingly.

This is where agentic workflows diverge from simple prompt chaining. A chain runs from A to B to C, regardless of context. An orchestrated workflow can branch, merge, retry, and adapt based on what agents actually return. For a deeper look at the structured approach to breaking down goals, see our guide on Planning Algorithms.

Task routing and agent coordination

Once tasks are decomposed, the orchestrator decides which agent, model, or tool handles each one. Routing decisions can depend on task type, agent capabilities, current load, access permissions, and runtime conditions. A software development workflow might route initial implementation to a coding agent and test generation to a testing agent, then pass the combined output to a reviewer.

Two protocols often sit underneath this. MCP (Model Context Protocol) standardizes how agents

access external resources such as databases, APIs, and file systems, whereas A2A (Agent-to-Agent) is one emerging protocol for peer-to-peer coordination between agents, rather than a universal standard. Routing logic sits above both: it decides which agent engages, while the protocols handle how agents communicate with each other and their tools.

State and context management

An orchestrator that loses track of intermediate results can't coordinate anything beyond a single call. State management is what gives a multistep workflow continuity.

The orchestration layer maintains several types of state simultaneously: the current task queue, intermediate outputs from completed steps, memory references for context-aware agents, and execution history for audit and retry logic. A coding agent that needs to understand the existing module structure before rewriting it will typically rely on the orchestrator to surface that context at execution time.

This connects directly to Memory in AI Agents, which covers how agents encode, retrieve, and use information across steps. The orchestrator coordinates when memory systems are read and written as the workflow progresses. A shared semantic context layer across repositories matters here, too. When agents work across different services, each one needs the same understanding of shared types, API contracts, and domain terms. Without that common reference, an agent editing a data model in one repository can generate changes

that break a consumer in another repository. A shared context layer gives every agent the same definitions to work against, so their independent changes stay compatible.

Feedback loops and workflow adaptation

Agents produce outputs that the orchestration system has to evaluate. Did the agent return a valid result, or did it time out, hallucinate a tool call, or return a partial response?

The orchestration layer checks each output against expected conditions and routes the workflow accordingly. On success, it updates the state and moves to the next task. On failure, it decides whether to retry, using the same or different parameters, take a fallback path, or escalate to a human. The limits that govern those decisions, including maximum retry counts, confidence thresholds, execution timeouts, and explicit stopping conditions, are the runtime controls that bound how far an agent runs on its own before it stops or hands back to a person. They prevent runaway loops, and they are where the orchestration layer deliberately keeps a human in the loop. Without this feedback mechanism, a single failed tool call can silently corrupt the entire workflow.

Core AI agent orchestration patterns

How coordination responsibility is assigned, whether to one controller, distributed across many components, or layered between them, determines a system's observability, fault tolerance, and scalability. These three AI orchestration patterns, centralized, distributed, and hierarchical, usually look interchangeable until you reach real production load, when the differences start to hurt. For a broader view of how orchestration fits into overall agent system design, see our guide on AI Agent Architecture.

Centralized orchestration

In centralized orchestration, a single orchestrator manages the full workflow: it maintains the task queue, routes every step, tracks all state, makes every scheduling decision, and prevents anything in the workflow from proceeding without it.

The main advantage is simplicity. When something breaks, you have one place to look, and all workflow states are visible in a single location, which makes observability and debugging straightforward. For linear pipelines, such as code generation followed by testing followed by review, this is often the right default.

The cost is scalability. That same single orchestrator is also a single point of failure, and under high concurrency or with long-running workflows, it becomes a bottleneck for systems that push many parallel agents through one coordinator.

Distributed orchestration

Distributed orchestration scales by giving up the single controller. Coordination spreads across multiple systems, queues, or agents that self-coordinate through shared messaging infrastructure: event queues, message brokers, and distributed state stores. Agents consume tasks from a queue independently; failures in one part of the system do not necessarily cascade to the rest, and no single node brings everything down. Large-scale parallel workloads, such as hundreds of concurrent agent tasks running across services, are a natural fit.

What you pay for that capacity is harder debugging and operational reasoning. Debugging a failure that spans three queues, two agent pools, and an external API takes more instrumentation than reading a single centralized log, and operational complexity climbs alongside throughput.

Distributed orchestration

Hierarchical orchestration organizes workflows through layered coordinator-worker relationships. A top-level orchestrator delegates to sub-orchestrators, which in turn manage specialized agents, and each level handles coordination at its own scope. A software development system might run a planning agent that decomposes a feature request into subtasks, a supervisor agent that assigns those subtasks to implementation, testing, and documentation agents, and individual agents that each run their own tool-calling loop internally. JetBrains Air⁷ is one implementation of this staged approach, moving an agent task through distinct planning, execution, and review stages with explicit handoffs between them.

The pattern balances control and specialization well, though that layering carries a latency cost: each coordination boundary adds overhead, and bugs can hide at the handoff between layers.

Orchestration in multi-agent systems

Orchestration gets more demanding when multiple specialized agents are involved. A single agent that fails mid-task is often recoverable. A workflow with three agents sharing state, depending on different upstream outputs, and one silently hanging is harder to diagnose and recover from.

Multi-agent systems⁷ introduce coordination requirements that don't exist in single-agent setups:

- **Task ownership:** Which agent is responsible for a given subtask? If two agents both attempt it, you get duplicate or conflicting outputs.
- **Shared context:** Agents working on the same codebase need a consistent world state. An agent unaware that another agent just refactored the auth module will generate incorrect changes.
- **Synchronization:** Tasks with inter-agent dependencies need explicit sequencing. Without it, an agent may start before its inputs are ready.
- **Handoff contracts:** Agents can lose context during handoff, return incompatible output formats, or enter infinite retry loops unless the orchestrator enforces contracts between them.

The orchestration layer meets these requirements by acting as the single source of truth for workflow state. It serializes conflicting writes, gates task execution on dependency completion, validates handoff schemas, and maintains the execution history that makes debugging tractable.

The same coordination layer is also where policy gets enforced as the agent count grows: deciding which models and agents are approved, applying usage rules, and keeping agent activity and cost visible across teams. That governance is something you either build into your own orchestration or get from a platform like JetBrains AI for Teams and Organization⁷, but architecturally it belongs at the coordination layer either way.

Common challenges in AI agent orchestration

The most stubborn AI agent orchestration challenges in production often come less from the agents themselves than from the coordination logic between them. Silent failures are the hardest category: an agent returns a partial result, the state gets written incorrectly, and the next agent proceeds on bad input with no error surfaced. Teams usually underestimate this until they are debugging a workflow that failed three hops ago. The table below maps the most common failure modes, why each one matters, and a practical mitigation approach.

Challenge	Why It Matters	Mitigation Approach
-----------	----------------	---------------------

State consistency	Concurrent agents can read or write conflicting workflow state	Use a centralized state store with atomic writes or optimistic locking
Synchronization complexity	Dependent tasks can deadlock or race if dependencies aren't explicit	Define dependency graphs before execution; gate tasks on predecessor completion
Workflow latency	Sequential bottlenecks compound across agent hops	Parallelize independent subtasks; set per-task execution timeouts
Debugging difficulty	Failures occur deep in multihop execution traces	Emit structured logs with correlation IDs across every agent call
Coordination failures	Handoffs drop context or create orphaned tasks	Enforce explicit handoff contracts; validate input and output schemas at handoff points
Permission management	Agents may access resources or models beyond their intended scope	Scope permissions per agent role; enforce access centrally
Scalability	Centralized orchestrators	Partition workflows; move high-

	become throughput bottlenecks under high concurrency	concurrency coordination toward distributed patterns
Observability	Workflow state and agent behavior are invisible without instrumentation	Emit spans and traces per task; LangSmith [↗] and OpenTelemetry [↗] help at scale

The mitigations in the table reduce these failures rather than eliminate them. These failure modes compound each other. Low observability makes debugging difficult, and poor debugging lets state consistency issues accumulate undetected. Addressing them early in the design process is far cheaper than untangling them once they interact in production.

Choosing the right AI agent orchestration strategy

There is no single default orchestration pattern. The right AI agent orchestration strategy depends on your workflow's complexity, concurrency requirements, and reliability targets, and most production systems end up combining approaches. If you're starting to build an agent system, see our guide on [How to Build an AI Agent for the foundational decisions that come before this one](#). Then weigh these factors:

- **Workflow complexity:** How many steps, branches, and agent types does the workflow

involve?

- **Agent count and concurrency:** How many agents run, and how many parallel tasks do you expect at peak load?
- **Latency budget:** How much per-hop overhead is acceptable? Each coordination layer and sequential dependency compounds end-to-end latency, with hierarchical patterns adding the most and distributed patterns reducing bottlenecks for parallel work.
- **Reliability targets:** What is the acceptable failure rate, and how fast does recovery need to be?
- **Observability needs:** How much visibility do you need into each step for debugging and auditing?
- **Runtime control:** How much execution control do you need while the workflow runs? Centralized patterns let you inspect and intervene at any step; distributed patterns trade some of that control for throughput.
- **Operational risk:** What happens if the orchestrator fails mid-workflow?

One common approach combines a centralized orchestrator for overall workflow management, distributed queues for parallel task execution, and, in larger deployments, hierarchical sub-orchestrators for specialized agent clusters.

Pattern	Strengths	Limitations	Ideal For
---------	-----------	-------------	-----------

Centralized	Easy to debug; consistent state; single point of control	Bottleneck at scale; single point of failure	Linear pipelines; audit-sensitive workflows
Distributed	High throughput; fault-tolerant; no central bottleneck	Complex to debug; harder to enforce ordering guarantees	Massively parallel workloads; independent agent tasks
Hierarchical	Balances control with specialization; maps well to complex objectives	Coordination overhead at each layer boundary; failures harder to trace	Planner-reviewer-executor systems; multiphase SDLC workflows

The most common mistake teams make is committing to a centralized orchestrator for simplicity, then discovering it doesn't scale and having to rearchitect under pressure. Build in observability from the start, and move toward distribution when you hit concrete performance bottlenecks rather than too early.

What this means for developers

Orchestration is the operational layer that makes agents useful at production scale. The gap between

an agent that works in a demo and one that runs reliably in a shared engineering workflow often comes down to how well the system's coordination logic is designed.

As agent systems take on more autonomy, always within the bounds a team sets, and spread across more services, orchestration complexity grows with them. A team running ten specialized agents across a software delivery lifecycle, including coding, testing, review, documentation, and deployment, can't rely on improvised coordination. The workflow logic has to be explicit, state management has to stay consistent, and failure handling has to be deliberate.

Those early decisions, including how coordination is distributed, how state is tracked, which instrument you use, and where the agent has to stop and ask, set the ceiling on a system's scalability, reliability, observability, and operational safety. The orchestration layer is where all four are enforced at runtime, which is why getting it right gives the agents on top of it room to do their work.

FAQ

How do orchestration systems coordinate AI agent execution flows?

The orchestrator maintains a task queue built from the decomposed objective. It evaluates each task's dependencies, routes each task to the appropriate agent or tool, and waits for a result before updating workflow state. After each step, it checks output validity and decides whether to proceed, retry, or escalate. The loop continues until the workflow

What happens when multiple AI agents depend on the same workflow state?

The orchestration layer manages write access to shared state, coordinating updates to prevent concurrent conflicts. In well-designed systems, agents don't write directly to a shared store simultaneously; the orchestrator serializes updates or uses atomic operations. When multiple agents need the same information as input, the orchestrator

reaches a terminal condition: success, failure, or escalation.

How do developers scale orchestration across large AI systems?

Scaling usually means moving from a single centralized orchestrator toward distributed coordination for high-concurrency tasks, while keeping centralized control for workflow-level decisions. In practice, that means using message queues or event streams (like Amazon SQS⁷ or Apache Kafka⁷) for task distribution, separate state stores for workflow tracking, and per-agent execution environments that scale independently. The orchestration layer itself stays thin, handling routing and policy, while the infrastructure underneath absorbs the throughput.

How do orchestration layers improve reliability in multi-agent workflows?

Orchestration improves reliability by making failures explicit and recoverable. The orchestrator catches failures at the boundary of each step and decides how to respond: retry with different parameters, route to a fallback agent, or surface the failure for human review. It also enforces handoff contracts between agents, which keeps downstream agents from operating on invalid inputs.

coordinates state access to keep it consistent, rather than letting agents read concurrently from a store that may be in the middle of an update.

What are the biggest failure points in AI agent orchestration?

The failures that hurt most are the silent ones. Explicit failures, such as timeouts, schema validation errors, and tool call exceptions, announce themselves and are straightforward to catch. Silent degradation is the hard case: it surfaces later in the workflow, far from the step that caused it, so the defenses have to be structural rather than reactive. Structured logging with correlation IDs, output validation at every handoff, and explicit stopping conditions matter more here than retry logic alone.

Damaso Sanoja is an engineer who is passionate about helping others make data-driven decisions to achieve their goals. This has motivated him to write numerous articles on the most popular relational databases, customer relationship management systems, enterprise resource planning systems, master data management tools, and, more recently, data warehouse systems used for machine learning and AI projects. You can blame this fixation on data management on his first computer being a Commodore 64 without a floppy disk.

Use AI agents in your development workflow

Explore JetBrains AI solutions that help you build, use, and scale AI agents across the software development lifecycle.

JetBrains Air

An open system of AI products for developers, teams, and organizations – from coding with agents to automating workflows and governing AI at scale.

[Explore Air](#)

Air in IDEs

Orchestrate AI agents and verify their output directly in JetBrains IDEs, with full control over how changes are reviewed.

Air Teams

Automate software delivery workflows, coordinate agentic work across teams, identify

Air Governance

Govern AI usage, models, policies, and costs across your organization, including JetBrains and third-party tools.

[Try Air in IDEs ↗](#)

bottlenecks
affecting delivery.

[Explore Air
Teams ↗](#)

[Explore Air
Governance ↗](#)

Air Context

Give agents shared organizational memory and context that carries across tools, workflows, and execution environments.

[Explore Air
Context ↗](#)

Air Gateway

Run terminal agents through one JetBrains account, with access, models, and usage governed centrally.

[Explore Air
Gateway ↗](#)

Junie

Plan, code, debug, and automate tasks with a coding agent that works across your terminal, IDE, GitHub, and GitLab.

[Try Junie ↗](#)

Continue Exploring the AI Agents for Developers Guide

What Are AI Agents?

Discover what AI agents are, how they work, their architecture, types, use cases, and how developers can build intelligent autonomous systems.

[Read more ↗](#)

Multi-Agent Systems for Developers

Learn how multi-agent systems coordinate AI agents, compare architecture patterns, solve complex workflows, and improve software development.

[Read more ↗](#)

What Are Autonomous AI Agents?

Learn what autonomous AI agents are, how they execute workflows, their levels of autonomy, key guardrails, risks, and software development use cases.

[Read more ↗](#)